RETAILMART V3 · SQL

Basic Queries & DDL Commands

— Siraj Sir

📱 WhatsApp Announcement

🚀 BASIC QUERIES & DDL COMMANDS 🚀

SQL Masterclass — Session 2
Join ON TIME from the portal: [INSERT LINK]

_____

🎬 Today's Focus:

- Writing your FIRST SQL queries!
- DDL Commands: CREATE, DROP, ALTER
- Building the RetailMart database
- Setting up schemas & importing real data!

Why This Matters:
Every database in the world starts with CREATE. Today you build your first one — the same way engineers at every company do on SQL Intro!

🔥 Pro Tip: CREATE TABLE is the single most important DDL command — you'll use it every day of your career!

📱 WhatsApp (continued)

📂 GitHub Repo: https://github.com/starlordali4444/PostgreSql
Let's build something REAL today! See you all ON TIME! ⚡
— Siraj Sir
#Day2 #PostgreSQL #DDLCommands

RECAP

🔄 YOUR SQL JOURNEY SO FAR

Introduction to SQL & Installation

- What is SQL? The universal language for relational databases
- SQL 5 categories: DDL, DML, DQL, DCL, TCL
- Database vs Schema: Building vs Departments inside
- Installed PostgreSQL 18, pgAdmin 4, VS Code with SQLTools
- Client-Server architecture: pgAdmin (Client) talks to PostgreSQL (Server) on Port 5432

🎯 Today (Setup & Onboarding)

- We're ready to CREATE our first database and write REAL SQL!

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: SQL Basics & SELECT	25 mins	Writing basic SQL queries, SELECT as a calculator, System info queries
Part 2: CREATE DATABASE & SCHEMA	25 mins	CREATE DATABASE, CREATE SCHEMA, IF NOT EXISTS pattern
Part 3: CREATE TABLE	30 mins	CREATE TABLE syntax, Column definitions & data types, Schema-qualified table names
Part 4: DROP & ALTER	30 mins	DROP DATABASE/SCHEMA/TABLE, IF EXISTS safety pattern, ALTER TABLE — ADD, DROP, RENAME columns
Part 5: Hands-On Lab	30 mins	Build RetailMart schemas, Create sample tables, CSV Import & Q&A

STORY

🎬 MAIN STORY: The School That Built Its Own Library System (1/2)

Duration: 10 minutes | No technical jargon — just the story!

① A small school has 500 students and a library with 5,000 books. The librarian, Mrs. Sharma, keeps everything in a thick register — handwritten entries for every book, every student, every borrow/return.

② One day, the principal asks: 'How many Science books do we have?' Mrs. Sharma spends 3 HOURS flipping through 200 pages, counting manually. The answer: 347.

③ The principal then asks: 'Which students have overdue books right now?' Another 2 hours of cross-referencing. Mrs. Sharma is exhausted.

④ A computer teacher offers to help. He says: 'First, I need a PLACE to store everything.' He creates a DATABASE — like buying a new empty filing cabinet.

STORY

🎬 MAIN STORY: The School That Built Its Own Library System (2/2)

⑤ Then he says: 'Inside this cabinet, I need SECTIONS.' He creates SCHEMAS — one section for 'books', one for 'students', one for 'borrowing records'. Like labeled drawers inside the cabinet.

⑥ Inside each section, he creates TABLES — like designing the columns on a new register page. The books table has columns for book_id, title, subject, author. The students table has student_id, name, class.

⑦ He types a few lines and suddenly, the answer to 'How many Science books?' appears in 0.01 seconds. Mrs. Sharma stares in disbelief.

⑧ 'That is SQL,' the teacher says. 'And those few lines I typed? CREATE DATABASE, CREATE SCHEMA, CREATE TABLE. That is exactly what we learn today.'

REVEAL

🔗 STORY → SQL CONNECTION (1/2)

Every part of the library story maps to today's SQL concepts:

- ① Handwritten register = Unstructured data (no database)
- ② 3 hours to count books = No query engine → SQL answers in milliseconds
- ③ Cross-referencing records = Manual lookups → Databases link tables automatically
- ④ Filing cabinet = CREATE DATABASE — the top-level container
- ⑤ Labeled drawers = CREATE SCHEMA — logical sections inside the database
- ⑥ Register columns = CREATE TABLE — defining the structure with columns and types
- ⑦ Instant answer = SELECT query — the power of structured data

REVEAL

🔗 STORY → SQL CONNECTION (2/2)

- ⑧ SQL commands = DDL (Data Definition Language) — building the structure

📱 WhatsApp Announcement

Setup & Onboarding! Yesterday you learned WHAT databases are. Today you'll learn to BUILD them. By the end of today, you'll CREATE databases, schemas, and tables — the same skill that database administrators at Amazon and Flipkart use daily. You're not just learning SQL — you're building the backbone of every application. Let's create something! 🏗️

"Knowing SQL theory is like reading a recipe. Today, you COOK." — Siraj Sir

STORY

🗣️ MINI STORY

Imagine you just bought a brand-new calculator. Before solving complex equations, you test it: 1 + 1 = ? If it says 2, you trust it.

SQL works the same way! Before building tables and querying millions of rows, you can use SELECT as a calculator, a clock, and a system inspector — all without needing a single table.

DEFINITION

📝 DEFINITION: SQL Query

🎓 **Technical:**

An SQL query is a structured instruction written in Structured Query Language that communicates with a relational database management system (RDBMS) to perform operations such as retrieving, inserting, updating, or deleting data. The most basic form is SELECT expression;

🗣️ **Layman's:**

Think of SQL like ordering food at a restaurant. You tell the waiter (database) exactly what you want: 'Give me today's date' or 'Calculate 100 times 18'. The waiter brings back exactly what you asked for. That request is a 'query'.

SYNTAX: Basic Queries

```
-- Basic structure
SELECT 42;

-- As a calculator
SELECT 1 + 1;                   -- Addition
SELECT 100 * 0.18;              -- Multiplication
SELECT 2025 - 1986;             -- Subtraction

-- System info (no FROM needed)
SELECT version();               -- PostgreSQL version
SELECT current_date;            -- Today's date
SELECT current_time;            -- Current time
SELECT current_timestamp;       -- Date + Time
SELECT current_user;            -- Who's logged in
SELECT current_database();      -- Which database

-- String operations
SELECT 'Hello' || ' ' || 'World!';  -- Concatenation
SELECT UPPER('hello');               -- HELLO
SELECT LENGTH('PostgreSQL');         -- 10
```

EASY

PROBLEM: You just installed PostgreSQL. You want to verify the installation is working before doing anything else.

🤔 Think about it: What's the simplest way to test if the database engine responds?

💡 What happens: PostgreSQL returns a long string showing the exact version, compiler, and platform. If you see this, your installation is healthy!

Solution

✅ ANSWER

```
SELECT version();
```

MEDIUM

PROBLEM: You want to check today's date, the current time, and which database you're connected to — all in one query.

🤔 Think about it: Can you combine multiple SELECTs into one?

💡 What happens: Returns one row with 4 columns. AS gives each column a friendly name (alias). This is your first multi-column SELECT!

Solution

✅ ANSWER

```
SELECT
  current_date AS today,
  current_time AS right_now,
  current_database() AS connected_to,
  current_user AS logged_in_as;
```

HARD

PROBLEM: You need a comprehensive system status report combining version, user, database, and timestamp in a single query.

🤔 Think about it: Can you combine system functions with calculations?

💡 What happens: Returns database name, user, full version string, exact timestamp, and your session's process ID. DBAs use this to identify sessions.

Solution

✅ ANSWER

```
SELECT
  current_database() AS database_name,
  current_user AS logged_in_user,
  version() AS postgres_version,
  current_timestamp AS report_generated_at,
  pg_backend_pid() AS session_process_id;
```

CRAZY

PROBLEM: Use SQL as a calculator to compute GST. A product costs ₹999 before tax. GST is 18%. What's the total?

🤔 Think about it: SQL can do math without any table!

💡 What happens: Returns 999, 179.82, 1178.82, 1178.82. ROUND() controls decimal places. Every billing system does this calculation!

Solution

✅ ANSWER

```
SELECT
  999 AS base_price,
  999 * 0.18 AS gst_amount,
  999 * 1.18 AS total_with_gst,
  ROUND(999 * 1.18, 2) AS rounded_total;
```

KEY TAKEAWAYS

SELECT is the most versatile SQL command. It works as a calculator, a clock, a system inspector, and (starting tomorrow) a data retriever. You don't always need a table — SELECT expression; works on its own.

PRO TIP

PRO TIP

Always end every SQL statement with a semicolon (;). PostgreSQL waits patiently for the semicolon before executing. Forgetting it is the #1 beginner mistake — your query appears to hang, but it's just waiting for you to finish!

COMMON MISTAKE

⚠ COMMON MISTAKE: Forgetting the Semicolon

❌ The Mistake:

Typing SELECT 1 + 1 and pressing Execute. Nothing happens. The cursor just moves to the next line.

✅ The Fix:

Every SQL statement MUST end with a semicolon: SELECT 1 + 1; — the semicolon tells PostgreSQL 'I'm done, execute this now!'

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'Can you run a SELECT statement without a FROM clause?'

A: 'Yes! In PostgreSQL, SELECT can be used as a calculator (SELECT 1+1;), to call system functions (SELECT version();), or to get current date/time (SELECT NOW();). No table is needed for these expressions.'

CHALLENGE

💻 TRY THIS

1.: Run SELECT version(); and note which PostgreSQL version you have.

2.: Calculate your age in days: SELECT current_date - '2000-01-15'; (use your birthday)

REVEAL

🧠 THINK ABOUT IT: The AS Keyword

You saw `AS today` in the examples. That's called a column alias. It doesn't change the stored data — it just renames the column in the OUTPUT.

Without alias: `SELECT current_date;` → column header appears as 'current_date'.
With alias: `SELECT current_date AS today;` → column header appears as 'today'.

The AS keyword is technically OPTIONAL: `SELECT current_date today;` also works. But ALWAYS use AS explicitly — it makes your SQL 10× more readable and it's an interview expectation. Never skip AS in production code.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between current_timestamp and NOW()?'
A: 'Functionally they are identical in PostgreSQL — both return the current date and time WITH timezone. current_timestamp is defined in the ANSI SQL standard (works on all databases). NOW() is PostgreSQL-specific and a little shorter to type. In portable code that may run on MySQL or Oracle, prefer current_timestamp. In PostgreSQL-only code, either is fine.'

PRO TIP

PRO TIP (1/2)

🧠 Pro Tip: SELECT Without a Table is a Secret Superpower

Most tutorials teach you that every SELECT needs a FROM clause. That's half-true — in PostgreSQL, you can `SELECT 1 + 1;` with no FROM and it works. Why? Because PostgreSQL has an invisible pseudo-table called the 'dual' concept under the hood.

Why is this useful? Because it turns SQL into a calculator that you can use for:

- Quick arithmetic when you don't have a Python interpreter open
- Testing functions BEFORE running them on real data (`SELECT UPPER('test');`)
- Checking system state (`SELECT version();`, `SELECT current_user;`)
- Generating dummy rows for testing (`SELECT generate_series(1, 10);` returns 10 rows!)

PRO TIP

PRO TIP (2/2)

Every senior SQL engineer uses this daily. Now you can too.

STORY

🗣️ MINI STORY

Imagine you want to open a new school. Before you can have classrooms, a library, or a playground — you need LAND. You need a building.

CREATE DATABASE is buying that land and constructing the empty building. Without it, there's nowhere to put your schemas (departments) or tables (registers). Everything starts here.

DEFINITION

📝 DEFINITION: CREATE DATABASE

🎓 **Technical:**

CREATE DATABASE is a DDL command that creates a new database in the PostgreSQL server. It allocates storage space and creates system catalogs (internal tables that track all objects). Each database is isolated — tables in one database cannot directly access tables in another.

🗣️ **Layman's:**

A database is like buying a new plot of land. Before you can build anything (tables, schemas), you need the land. Each plot (database) is fenced off — what's inside one plot is completely separate from another.

SYNTAX: CREATE DATABASE

```
-- Basic syntax
CREATE DATABASE database_name;

-- With options
CREATE DATABASE database_name
  OWNER = username
  ENCODING = 'UTF8'
  LC_COLLATE = 'en_US.UTF-8'
  LC_CTYPE = 'en_US.UTF-8';

-- Check existing databases
SELECT datname FROM pg_database
  WHERE datistemplate = false;
```

EASY

PROBLEM: You're starting a new project and need a fresh database to work in.

🤔 Think about it: What's the very first SQL command you run?

💡 What happens: PostgreSQL creates an empty database. You can now connect to it and start building schemas and tables inside.

Solution

✅ ANSWER

```
CREATE DATABASE my_first_db;
```

MEDIUM

PROBLEM: You need to create the RetailMart practice database and verify it was created.

🤔 Think about it: How do you confirm the database exists after creating it?

💡 What happens: pg_database is PostgreSQL's internal catalog. If the query returns a row, your database exists!

Solution

✅ ANSWER

```
CREATE DATABASE retailmart_practice;

-- Verify it was created
SELECT datname FROM pg_database
  WHERE datname = 'retailmart_practice';
```

HARD

PROBLEM: You need separate databases for development, testing, and staging environments.

🤔 Think about it: Why would you need multiple databases?

💡 What happens: Three isolated databases are created. A bug in dev can never corrupt staging data. This is standard practice at every company.

Solution

✅ ANSWER

```
CREATE DATABASE retailmart_dev;
CREATE DATABASE retailmart_test;
CREATE DATABASE retailmart_staging;

-- Verify all three
SELECT datname FROM pg_database
  WHERE datname LIKE 'retailmart_%';
```

CRAZY

PROBLEM: List ALL non-template databases on your server with their sizes and owners.

🤔 Think about it: PostgreSQL stores metadata about itself in system catalogs!

🔍 What happens: Returns every database with its owner and human-readable size (like '8 MB'). Senior DBAs run this daily to monitor disk usage.

Solution

✅ ANSWER

```
SELECT
  datname AS database_name,
  pg_catalog.pg_get_userbyid(datdba) AS owner,
  pg_catalog.pg_database_size(datname) AS size_bytes,
  pg_catalog.pg_size_pretty(
    pg_catalog.pg_database_size(datname)
  ) AS size_human
FROM pg_database
WHERE datistemplate = false
ORDER BY size_bytes DESC;
```

KEY TAKEAWAYS

CREATE DATABASE is always the first step. Each database is isolated. Use pg_database to inspect what exists. In production, you typically have separate databases for dev, test, staging, and production environments.

PRO TIP

PRO TIP

You CANNOT run CREATE DATABASE while connected to the database you're creating. You must be connected to a different database (usually 'postgres', the default). In pgAdmin, right-click 'Databases' → 'Create' → 'Database' for a GUI approach.

COMMON MISTAKE

⚠️ COMMON MISTAKE: Creating Database While Connected To It

❌ The Mistake:

Trying to DROP or CREATE a database you're currently connected to. PostgreSQL says 'cannot drop the currently open database'.

✅ The Fix:

Always connect to the 'postgres' default database first, then run your CREATE/DROP DATABASE commands from there.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What happens internally when you run CREATE DATABASE?'

A: 'PostgreSQL copies the template database (template1) to create a new, empty database. It allocates a new directory on disk, creates system catalogs (pg_class, pg_attribute, etc.), and registers the new database in pg_database. The new database is completely isolated from others.'

CHALLENGE

CHALLENGE

💻 TRY THIS

1.: Create a database called day2_practice.

2.: Verify it exists using SELECT datname FROM pg_database;

REAL WORLD

REAL WORLD (1/2)

🏢 DEV / TEST / STAGING / PROD — The 4-Environment Pattern

Every serious tech company runs FOUR separate databases for the same product:

- `retailmart_dev` — developers break things freely, reset anytime
- `retailmart_test` — automated tests run against isolated data
- `retailmart_staging` — mirrors production; final rehearsal before release
- `retailmart_prod` — THE real database; real customers, real money

REAL WORLD

REAL WORLD (2/2)

Each environment is a separate CREATE DATABASE. Same schema, same tables — different data, different risk levels. A junior engineer's typo in dev is a shrug. The same typo in prod is a Slack page at 2 AM. That's why this separation exists. You'll see this pattern at every company you work at for the rest of your career.

PRO TIP

PRO TIP (1/2)

🧠 Pro Tip: Naming Conventions for Databases

Good database names save your future self hours of confusion. Follow these industry patterns:

- ✅ `retailmart_prod`, `retailmart_staging`, `retailmart_dev` (environment suffix)
- ✅ `analytics_warehouse`, `orders_operational` (purpose in the name)
- ❌ `db1`, `test`, `new_db`, `final_final_db` (unclear, embarrassing in meetings)
- ❌ `MyCompanyDatabase` (mixed case — PostgreSQL lowercases it anyway)

PRO TIP

PRO TIP (2/2)

Stick to snake_case. Always lowercase. Suffix with environment. Never use generic names. A good database name is self-documenting.

STORY

🗣️ MINI STORY

Your school building is ready (the database). Now you need DEPARTMENTS inside it. The Science Wing, the Arts Wing, the Admin Block, and the Sports Complex.

Without departments, every classroom is just scattered randomly. With departments, teachers know exactly where to go: 'Physics lab? Science Wing, Room 3.'

That's what a SCHEMA does — it organizes your tables into logical groups inside the database.

DEFINITION

📝 DEFINITION: CREATE SCHEMA

🎓 **Technical:**

CREATE SCHEMA creates a namespace within a database to logically group related database objects (tables, views, functions). Schemas provide organization, prevent naming conflicts, and help manage access permissions. Every database has a default schema called 'public'.

🗣️ **Layman's:**

If the DATABASE is your office building, SCHEMA is like different departments inside. Just like a company has HR, Finance, and Sales departments, a database has schemas to organize tables into logical groups.

SYNTAX: CREATE SCHEMA

```
-- Basic syntax
CREATE SCHEMA schema_name;

-- Safe syntax (won't error if exists)
CREATE SCHEMA IF NOT EXISTS schema_name;

-- Verify schemas
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name NOT IN ('pg_catalog',
  'information_schema', 'pg_toast');

-- Access a table inside a schema
SELECT * FROM schema_name.table_name;
```

EASY

PROBLEM: You need to create a 'students' section in your library database.

🤔 Think about it: How do you create a logical section inside a database?

💡 What happens: A namespace called 'students' is created. Any table you create inside it will be accessed as students.table_name.

Solution

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS students;
```

MEDIUM

PROBLEM: RetailMart V2 needs organized sections for customers, products, and sales data.

🤔 Think about it: What logical groupings does an e-commerce company need?

💡 What happens: Four schemas are created. IF NOT EXISTS means running this script twice causes no errors — it simply skips existing schemas.

Solution

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS customers;
CREATE SCHEMA IF NOT EXISTS products;
CREATE SCHEMA IF NOT EXISTS sales;
CREATE SCHEMA IF NOT EXISTS core;

-- Verify all schemas
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN (
  'customers', 'products', 'sales', 'core'
);
```

HARD

PROBLEM: Set up the complete RetailMart V2 schema architecture with all 8 core schemas.

🤔 Think about it: A real e-commerce platform needs many departments!

💡 What happens: 8 schemas created, each acting as a logical department. The count query confirms all were created successfully.

Solution

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS customers;
CREATE SCHEMA IF NOT EXISTS products;
CREATE SCHEMA IF NOT EXISTS sales;
CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS stores;
CREATE SCHEMA IF NOT EXISTS support;
CREATE SCHEMA IF NOT EXISTS marketing;
CREATE SCHEMA IF NOT EXISTS analytics;

-- Count schemas
SELECT COUNT(*) AS schema_count
FROM information_schema.schemata
WHERE schema_name NOT IN (
  'pg_catalog', 'information_schema',
  'pg_toast', 'public'
);
```

CRAZY

PROBLEM: List all schemas with the number of tables inside each one.

🤔 Think about it: How does PostgreSQL track which tables belong to which schema?

💡 What happens: Shows each schema and how many tables it contains. Don't worry about GROUP BY yet — we'll master it on Aggregates & Grouping. For now, just know this is how DBAs audit schema organization.

Solution

✅ ANSWER

```
SELECT
  schemaname AS schema_name,
  COUNT(*) AS table_count
FROM pg_catalog.pg_tables
WHERE schemaname NOT IN (
  'pg_catalog', 'information_schema'
)
GROUP BY schemaname
ORDER BY table_count DESC;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

Schemas are the organizational backbone of a database. They group related tables, prevent naming conflicts (two schemas can each have a 'users' table), and enable permission control. Always use IF NOT EXISTS for idempotent scripts.

PRO TIP

PRO TIP

Always use schema-qualified names (schema_name.table_name) instead of relying on the default 'public' schema. This makes your SQL self-documenting and prevents accidental cross-schema confusion. In RetailMart V2, we always write customers.customers, not just customers.

COMMON MISTAKE

❓❓❓ COMMON MISTAKE: Putting Everything in 'public'

❌ The Mistake:

Creating all tables in the default 'public' schema. After 50+ tables, you can't tell which tables are related to customers vs products vs sales.

✅ The Fix:

Always create dedicated schemas FIRST, then create tables inside them. This is how every production database is organized.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between a Database and a Schema?'

A: 'A Database is the top-level container — like a building. A Schema is a namespace inside the database — like departments within the building. Tables live inside Schemas. You connect to a Database; you organize with Schemas. The hierarchy is: Server → Database → Schema → Table.'

CHALLENGE

CHALLENGE

💻 TRY THIS

1.: Create 3 schemas: library, staff, records.

2.: Verify them with information_schema.schemata.

REVEAL

🧠 THINK ABOUT IT: The Mysterious 'public' Schema

Every PostgreSQL database ships with a default schema called `public`. It's created automatically and sits at the top of your SEARCH_PATH (PostgreSQL's way of saying 'where to look first').

When you write `CREATE TABLE customers (...);` WITHOUT a schema prefix, PostgreSQL drops it into `public` by default. This is convenient for tutorials — but a NIGHTMARE in real systems.

Why? Because after 6 months of careless work, `public` becomes a graveyard of 200+ unrelated tables — some real, some test, some experimental. Nobody remembers which is which. Morale collapses. Teams rage-quit.

The fix: NEVER use `public` for real tables. Treat it like a dirty hallway where you drop your backpack for 10 seconds. Build proper schemas (customers, products, sales) and put your tables THERE. Your future team will thank you.

COMMON MISTAKE

⚠️ COMMON MISTAKE: Schema Search Path Confusion

❌ The Mistake:

You create `CREATE TABLE customers.customers (...);` then write `SELECT * FROM customers;` and get ERROR: 'relation "customers" does not exist'.

✅ The Fix:

PostgreSQL only auto-finds tables in schemas on your SEARCH_PATH (by default just `public`). For any non-public schema, you MUST qualify: `SELECT * FROM customers.customers;`. Alternative: run `SET search_path TO customers, public;` at session start to include your schema in the auto-lookup.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'Can two tables in the same database have the same name?'

A: 'Yes — as long as they are in different schemas. For example, you can have `customers.orders` and `sales.orders` coexisting in the same database without conflict. Schemas act as namespaces. Fully qualifying the table name (schema.table) always removes ambiguity. This is one of the main reasons schemas exist — they prevent naming collisions in large systems with hundreds of tables.'

STORY

🗣️ MINI STORY

The school building is built (database). The departments exist (schemas). Now the class teacher needs to create a CLASS REGISTER — a blank register with specific columns: Roll Number, Student Name, Date of Birth, Marks.

She decides the columns FIRST, then students fill in their rows. If she makes Roll Number a text field and someone writes 'ABC', the register becomes useless. Types matter!

CREATE TABLE is designing that blank register — deciding what columns exist, what type of data each column holds, and in which schema (department) the register belongs.

DEFINITION

📝 DEFINITION: CREATE TABLE

🎓 **Technical:**

CREATE TABLE defines a new table structure with columns, data types, and optional constraints. Tables are the fundamental storage unit in relational databases. Each column has a name and a strict data type (INT, VARCHAR, DATE, etc.) that PostgreSQL enforces on every INSERT.

🗣️ **Layman's:**

A table is like an Excel spreadsheet with strict rules. CREATE TABLE is designing the column headers and deciding what kind of data each column accepts. Once created, every row must follow the blueprint. No exceptions.

SYNTAX: CREATE TABLE (1/2)

```
-- Basic syntax
CREATE TABLE schema_name.table_name (
  column1  DATA_TYPE,
  column2  DATA_TYPE,
  column3  DATA_TYPE
);

-- Safe syntax
CREATE TABLE IF NOT EXISTS schema_name.table_name (
  column1  DATA_TYPE,
  column2  DATA_TYPE
);

-- Common data types (DDL & Data Types deep dive)
-- INT            → whole numbers
-- SERIAL         → auto-increment integer
-- VARCHAR(n)     → text up to n characters
-- TEXT            → unlimited text
-- NUMERIC(p,s)   → exact decimal
-- DATE            → date only
```

SYNTAX: CREATE TABLE (2/2)

```
-- TIMESTAMPTZ     → date + time + timezone
-- BOOLEAN         → true/false
```

EASY

PROBLEM: Create a simple table to store book information for the school library.

🤔 Think about it: What columns does a book need? An ID, a title, an author, and a subject.

💡 What happens: An empty table with 4 columns is created. SERIAL means book_id auto-increments (1, 2, 3...) with every new row.

Solution

✅ ANSWER

```
CREATE TABLE IF NOT EXISTS public.books (
  book_id    SERIAL,
  title      VARCHAR(200),
  author     VARCHAR(100),
  subject    VARCHAR(50)
);
```

MEDIUM

PROBLEM: Create a customers table inside the customers schema for RetailMart.

🤔 Think about it: What information does a retail company need about its customers?

💡 What happens: Table created inside the customers schema. DEFAULT NOW() means created_at auto-fills with the current timestamp when a row is inserted.

Solution

✅ ANSWER

```
CREATE TABLE IF NOT EXISTS customers.demo_customers (
  customer_id   SERIAL,
  first_name    VARCHAR(50),
  last_name     VARCHAR(50),
  email         VARCHAR(150),
  phone         VARCHAR(15),
  created_at    TIMESTAMPTZ DEFAULT NOW()
);
```

HARD

PROBLEM: Create a products table with various data types showing price, stock, and active status.

🤔 Think about it: Products need numbers (price), text (name), booleans (active), and dates (launch date).

💡 What happens: NUMERIC(10,2) means up to 10 digits total with 2 decimal places — perfect for money. DEFAULT values auto-fill when not provided.

Solution

✅ ANSWER

```
CREATE TABLE IF NOT EXISTS products.demo_products (
  product_id    SERIAL,
  product_name  VARCHAR(200),
  brand         VARCHAR(100),
  price         NUMERIC(10, 2),
  stock_qty     INT DEFAULT 0,
  is_active     BOOLEAN DEFAULT TRUE,
  launch_date   DATE
);
```

CRAZY

PROBLEM: Create an orders table that tracks who ordered what, when, and for how much — showing how tables relate to each other conceptually.

🤔 Think about it: An order references a customer. How do we connect them?

💡 What happens: The orders table stores a customer_id that conceptually points to the customers table. On Constraints & DML, we'll make this connection official with FOREIGN KEYs. The information_schema query shows the table's blueprint.

Solution

✅ ANSWER

```
CREATE TABLE IF NOT EXISTS sales.demo_orders (
  order_id      SERIAL,
  customer_id   INT,          -- references customers
  order_date    TIMESTAMPTZ DEFAULT NOW(),
  status        VARCHAR(20) DEFAULT 'pending',
  net_total     NUMERIC(12, 2)
);

-- Verify the table structure
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'sales'
  AND table_name = 'demo_orders'
ORDER BY ordinal_position;
```

KEY TAKEAWAYS

CREATE TABLE is the workhorse of DDL. Every column needs a name and a data type. Use schema-qualified names (schema.table). Use IF NOT EXISTS for safe, repeatable scripts. DEFAULT values reduce the burden on INSERT statements.

PRO TIP

PRO TIP

Always use IF NOT EXISTS when writing CREATE TABLE in scripts. This makes your SQL scripts idempotent — you can run them 10 times and get the same result without errors. Production deployment scripts MUST be idempotent.

COMMON MISTAKE

⁉️ COMMON MISTAKE: Forgetting the Schema Name

❌ The Mistake:

Writing CREATE TABLE customers (...) instead of CREATE TABLE customers.customers (...). The table ends up in the 'public' schema, not the 'customers' schema.

✅ The Fix:

Always use the full schema.table_name format: CREATE TABLE customers.customers (...). This is explicit and prevents confusion.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between CREATE TABLE and CREATE TABLE IF NOT EXISTS?'

A: 'CREATE TABLE throws an error if the table already exists. CREATE TABLE IF NOT EXISTS silently does nothing if the table exists, and creates it only if it does not. The IF NOT EXISTS variant is essential for deployment scripts that might run multiple times.'

CHALLENGE

CHALLENGE

💻 TRY THIS

1.: Create a table stores.demo_stores with: store_id SERIAL, store_name VARCHAR(100), city VARCHAR(50), opening_date DATE (matches RetailMart V2's real stores.stores column name).

2.: Verify it with information_schema.columns.

REAL WORLD (1/4)

🏢 HOW REAL COMPANIES WRITE CREATE TABLE

A production-grade CREATE TABLE statement at companies like Flipkart or Razorpay includes FAR more than just column definitions. Here is a realistic example:

```
[Production CREATE TABLE — Example (matches our real RetailMart V2 sales.orders)]
CREATE TABLE IF NOT EXISTS sales.orders (
  order_id         INT PRIMARY KEY,
  cust_id          INT NOT NULL
    REFERENCES customers.customers(customer_id),
```

REAL WORLD (2/4)

```
store_id         INT NOT NULL
  REFERENCES stores.stores(store_id),
order_date       DATE NOT NULL DEFAULT CURRENT_DATE,
order_status     VARCHAR(20) NOT NULL
  DEFAULT 'Processing'
  CHECK (order_status IN (
    'Processing','Shipped','Out for Delivery',
    'Delivered','Cancelled','Returned','Failed'
  )),
```

REAL WORLD (3/4)

```
  gross_total      NUMERIC(12,2) NOT NULL CHECK (gross_total >= 0),
  discount_amount  NUMERIC(12,2) NOT NULL DEFAULT 0
    CHECK (discount_amount >= 0),
  net_total        NUMERIC(12,2) NOT NULL CHECK (net_total >= 0)
);


CREATE INDEX idx_orders_cust_id  ON sales.orders(cust_id);
CREATE INDEX idx_orders_store_id ON sales.orders(store_id);
CREATE INDEX idx_orders_date     ON sales.orders(order_date DESC);
```

REAL WORLD

REAL WORLD (4/4)

Notice: NOT NULL, DEFAULT, PRIMARY KEY, REFERENCES (foreign key), CHECK constraints, and follow-up CREATE INDEX statements. The CHECK on order_status uses the EXACT 7 values that appear in our RetailMart CSV data — Processing, Shipped, Out for Delivery, Delivered, Cancelled, Returned, Failed. We learn constraints on Constraints & DML and indexes on Query Plans — by then you'll be writing table definitions like this in your sleep.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between VARCHAR and TEXT in PostgreSQL?'
A: 'Functionally they are almost identical in PostgreSQL — both store variable-length strings. VARCHAR(n) enforces a maximum length of n characters; TEXT has no length limit. Performance-wise, there is NO speed difference — PostgreSQL internally uses the same storage for both (a hidden TOAST mechanism). In other databases like MySQL, VARCHAR has limits and TEXT is stored off-row, so the distinction matters. In PostgreSQL, many engineers default to TEXT for simplicity and use VARCHAR(n) only when they genuinely need the length constraint (e.g., phone numbers limited to 15 chars).'

PRO TIP

PRO TIP (1/2)

🧠 Pro Tip: Order of Columns Matters (Slightly)

When designing a CREATE TABLE, put columns in this order for maximum readability:

- 1. Primary Key (always first) — `id SERIAL PRIMARY KEY`
- 2. Foreign Keys / IDs — `customer_id`, `product_id`
- 3. Core business fields — `name`, `price`, `status`
- 4. Optional/descriptive fields — `notes`, `metadata`
- 5. Audit columns (always last) — `created_at`, `updated_at`, `deleted_at`

PRO TIP

PRO TIP (2/2)

This convention is followed by 90% of well-designed schemas. When someone reads your CREATE TABLE for the first time, their eyes scan top-to-bottom — the most important info should hit them first.

STORY

🗣️ MINI STORY

Imagine your school decides to demolish the old Sports Complex to build a new one. The demolition crew doesn't carefully remove each piece of furniture — they bring a wrecking ball and the entire building is GONE. Classrooms, equipment, trophies — everything inside, destroyed.

DROP is that wrecking ball. It doesn't just empty a table — it removes the table, its data, its structure, and its index completely. There's no undo button.

DEFINITION

??? DEFINITION: DROP

🎓 **Technical:**

DROP permanently removes database objects (tables, schemas, databases) and all data within them. Unlike DELETE (which removes rows), DROP removes the entire structure. It is DDL — irreversible without a backup.

🗣️ **Layman's:**

DROP is like demolishing a building. DELETE is like moving furniture out. With DROP, the building itself is gone — walls, roof, everything. You'd need to rebuild from scratch.

SYNTAX: DROP

```
-- Drop a table
DROP TABLE table_name;
DROP TABLE IF EXISTS table_name;
DROP TABLE IF EXISTS schema.table_name;

-- Drop a schema (must be empty)
DROP SCHEMA schema_name;
DROP SCHEMA IF EXISTS schema_name;

-- Drop a schema AND everything inside it
DROP SCHEMA IF EXISTS schema_name CASCADE;

-- Drop a database
DROP DATABASE IF EXISTS database_name;
```

EASY

PROBLEM: You created a test table while experimenting and want to remove it.

🤔 Think about it: How do you safely remove a table that might or might not exist?

💡 What happens: If the table exists, it's destroyed completely. If it doesn't exist, no error — the command silently succeeds.

Solution

✅ ANSWER

```
DROP TABLE IF EXISTS public.test_table;
```

MEDIUM

PROBLEM: You need to remove multiple test tables you created during practice.

🤔 Think about it: Can you drop multiple tables in one command?

💡 What happens: All three tables are removed in a single command. The verification query confirms they no longer exist.

Solution

✅ ANSWER

```
-- Drop multiple tables at once
DROP TABLE IF EXISTS
  public.test1,
  public.test2,
  public.test3;

-- Verify they're gone
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

HARD

PROBLEM: You need to remove an entire schema that has tables inside it.

🤔 Think about it: Can you drop a schema that still contains tables?

💡 What happens: DROP SCHEMA without CASCADE only works on empty schemas. CASCADE is the wrecking ball — it destroys the schema AND all tables, views, and functions inside it.

Solution

✅ ANSWER

```
-- Create a test schema with tables
CREATE SCHEMA IF NOT EXISTS temp_project;
CREATE TABLE temp_project.users (id SERIAL);
CREATE TABLE temp_project.logs (id SERIAL);

-- This FAILS — schema is not empty
DROP SCHEMA temp_project;
-- ERROR: schema is not empty

-- This WORKS — CASCADE destroys everything inside
DROP SCHEMA IF EXISTS temp_project CASCADE;
```

CRAZY

PROBLEM: You want to create a 'clean start' script that drops and recreates a schema from scratch.

🤔 Think about it: How do professionals reset a development environment?

💡 What happens: The schema is destroyed and recreated fresh. This 'drop-and-recreate' pattern is used daily in development environments to reset to a clean state.

Solution

✅ ANSWER

```
-- Clean start pattern (common in dev environments)
DROP SCHEMA IF EXISTS sandbox CASCADE;
CREATE SCHEMA sandbox;

-- Now create fresh tables
CREATE TABLE sandbox.test_customers (
  id SERIAL, name VARCHAR(100)
);
CREATE TABLE sandbox.test_orders (
  id SERIAL, amount NUMERIC(10,2)
);

-- Verify
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'sandbox';
```

KEY TAKEAWAYS

DROP is permanent and irreversible. Always use IF EXISTS to prevent errors. CASCADE drops everything inside a schema/table. Never run DROP on production without triple-checking your connection. The IF EXISTS pattern makes scripts safe to run multiple times.

PRO TIP

PRO TIP

Before running any DROP command, run a SELECT to see what you're about to destroy. The 2-second SELECT check can save you from a 2-week disaster recovery.

COMMON MISTAKE

⚠️ COMMON MISTAKE: DROP on the Wrong Database

❌ The Mistake:

Running DROP TABLE orders; while connected to the PRODUCTION database instead of the test database. The orders table — with 500,000 real customer orders — is gone instantly.

✅ The Fix:

ALWAYS check current_database() before any DROP command. In pgAdmin, the database name is visible in the tab title. Make it a habit: SELECT current_database(); before DROP anything.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between DROP TABLE and TRUNCATE TABLE?'

A: 'DROP TABLE removes the table structure AND all its data — the table ceases to exist. TRUNCATE TABLE removes all rows but keeps the table structure intact, ready for new data. DROP is like demolishing a building; TRUNCATE is like emptying all the furniture but leaving the building standing.'

CHALLENGE

CHALLENGE

💻 **TRY THIS**

1.: Create a schema 'playground' with 2 tables inside it.

2.: Try DROP SCHEMA playground; (observe the error).

3.: Then use DROP SCHEMA playground CASCADE; and verify it's gone.

STORY

🗣️ MINI STORY

Your school's class register was designed in January with columns: Roll No, Name, Marks. But in March, the principal says: 'We also need a Date of Birth column!'

You don't throw away the entire register and start over. You simply ADD a new column to the existing register. That's ALTER TABLE — modifying the structure without destroying the data.

ALTER is like renovating a house — adding a room, widening a door, renaming a room — all without demolishing the building.

DEFINITION

📝 DEFINITION: ALTER TABLE

🎓 **Technical:**

ALTER TABLE modifies an existing table's structure without losing data. It can add columns, drop columns, rename columns, change data types, set defaults, and add/remove constraints. It is a DDL command.

🗣️ **Layman's:**

ALTER TABLE is like renovating a house. You can add a new room (ADD COLUMN), remove a wall (DROP COLUMN), rename a room (RENAME COLUMN), or change the flooring material (ALTER COLUMN TYPE) — all without demolishing the building or losing what's inside.

SYNTAX: ALTER TABLE (1/2)

```
-- Add a new column
ALTER TABLE schema.table_name
  ADD COLUMN column_name DATA_TYPE;

-- Drop a column
ALTER TABLE schema.table_name
  DROP COLUMN column_name;

-- Rename a column
ALTER TABLE schema.table_name
  RENAME COLUMN old_name TO new_name;

-- Change data type
ALTER TABLE schema.table_name
  ALTER COLUMN column_name TYPE NEW_DATA_TYPE;

-- Set / remove default
ALTER TABLE schema.table_name
  ALTER COLUMN column_name SET DEFAULT value;
ALTER TABLE schema.table_name
```

SYNTAX: ALTER TABLE (2/2)

```
  ALTER COLUMN column_name DROP DEFAULT;

-- Rename the table itself
ALTER TABLE schema.old_table RENAME TO new_table;
```

EASY

PROBLEM: Your customers table needs a phone column that wasn't included in the original design.

🤔 Think about it: How do you add a column to a table that already has data?

💡 What happens: A new column 'phone' is added. Existing rows get NULL in this column. No data is lost!

Solution

✅ ANSWER

```
ALTER TABLE customers.demo_customers
  ADD COLUMN phone VARCHAR(15);
```

MEDIUM

PROBLEM: The products table needs multiple changes: add a 'category' column and rename 'brand' to 'brand_name'.

🤔 Think about it: Can you make multiple changes to the same table?

💡 What happens: Both changes are applied. The old data in 'brand' is preserved under the new name 'brand_name'. Category starts as NULL for all existing rows.

Solution

✅ ANSWER

```
-- Add category column
ALTER TABLE products.demo_products
  ADD COLUMN category VARCHAR(50);

-- Rename brand to brand_name
ALTER TABLE products.demo_products
  RENAME COLUMN brand TO brand_name;
```

HARD

PROBLEM: The phone column was created as VARCHAR(15) but international numbers need VARCHAR(20). Change the type.

🤔 Think about it: Can you change a column's data type after creation?

💡 What happens: The column type is expanded from VARCHAR(15) to VARCHAR(20). Existing data is preserved. Expanding VARCHAR is always safe; shrinking can fail if existing data is too long.

Solution

✅ ANSWER

```
-- Expand VARCHAR length
ALTER TABLE customers.demo_customers
  ALTER COLUMN phone TYPE VARCHAR(20);

-- Verify the change
SELECT column_name, data_type,
  character_maximum_length
FROM information_schema.columns
WHERE table_schema = 'customers'
  AND table_name = 'demo_customers'
  AND column_name = 'phone';
```

CRAZY

PROBLEM: You need to remove an obsolete column and add a default to an existing column — all in one go.

🤔 Think about it: How many alterations can you chain together?

💡 What happens: The launch_date column is removed (DROP COLUMN IF EXISTS prevents errors if already removed). is_active gets a default. A new updated_at column is added with auto-timestamp. The final query shows the full table blueprint.

Solution

✅ ANSWER

```
-- Remove obsolete column
ALTER TABLE products.demo_products
  DROP COLUMN IF EXISTS launch_date;

-- Add default to is_active
ALTER TABLE products.demo_products
  ALTER COLUMN is_active SET DEFAULT TRUE;

-- Add and set default in one statement
ALTER TABLE products.demo_products
  ADD COLUMN updated_at TIMESTAMPTZ
  DEFAULT NOW();

-- Verify final structure
SELECT column_name, data_type, column_default
FROM information_schema.columns
WHERE table_schema = 'products'
  AND table_name = 'demo_products'
ORDER BY ordinal_position;
```

KEY TAKEAWAYS

ALTER TABLE is the safe way to evolve your database over time. ADD COLUMN for new requirements, DROP COLUMN for obsolete fields, RENAME COLUMN for clarity, ALTER COLUMN TYPE for expanding capacity. Never use DROP TABLE + CREATE TABLE when ALTER TABLE will do.

PRO TIP

PRO TIP

In production, ALTER TABLE commands are tracked in 'migration' files — versioned SQL scripts that record every structural change. This ensures your dev, staging, and production databases stay in sync. Tools like Flyway and Alembic automate this process.

COMMON MISTAKE

⁉️ COMMON MISTAKE: Shrinking a Column Type

❌ The Mistake:

Changing VARCHAR(200) to VARCHAR(50) when existing data has strings longer than 50 characters. PostgreSQL rejects the change: 'value too long for type character varying(50)'.

✅ The Fix:

Always check the maximum length of existing data BEFORE shrinking: SELECT MAX(LENGTH(column_name)) FROM table; Only shrink if the max is within the new limit.

INTERVIEW

🎯 INTERVIEW SPOTLIGHT

Q: 'What is the difference between DROP TABLE and ALTER TABLE DROP COLUMN?'

A: 'DROP TABLE removes the entire table — structure and all data gone. ALTER TABLE DROP COLUMN removes just one column from the table while keeping all other columns and data intact. DROP TABLE is demolition; ALTER TABLE DROP COLUMN is surgical removal.'

CHALLENGE

CHALLENGE (1/2)

💻 **TRY THIS**

1.: Create a table public.test_alter with columns: id SERIAL, name VARCHAR(50).

2.: ADD COLUMN email VARCHAR(100).

3.: RENAME COLUMN name TO full_name.

4.:

CHALLENGE

CHALLENGE (2/2)

Verify with information_schema.columns.

REAL WORLD

REAL WORLD

In RetailMart V2, the database has 16 schemas and 47 tables. Each schema groups related tables: customers schema has customer data, products schema has product catalogs, sales schema has orders and payments. This organization is identical to how every major e-commerce platform structures their databases. You just learned the exact commands used to build it!

REAL WORLD

REAL WORLD

🎯 WHAT YOU ACHIEVED TODAY

You can now CREATE databases, schemas, tables — and ALTER or DROP them. You've written DDL commands that mirror exactly what happens when a startup builds its first database or a company adds a new feature. These are foundational skills used in EVERY tech company.

Tomorrow you'll master Data Types — learning to pick the PERFECT type for every column. It's like choosing the right tool for every job.

"Setup & Onboarding complete. You went from knowing SQL to DOING SQL. That's a bigger leap than most people realize." — Siraj Sir

DEFINITION

The 5-Question DDL Decision Tree (1/3)

Before typing any DDL command, walk through these 5 questions — in order. If you answer 'yes' to any, stop and use the command for that step.

Q1 — Does the object exist yet?

→ No? → CREATE. Always use IF NOT EXISTS so re-runs don't crash.

Q2 — Is the object's PURPOSE changing entirely (e.g., 'orders' → 'invoices')?

→ Yes? → RENAME (ALTER TABLE … RENAME TO). Never DROP + CREATE — you'll lose the data.

Q3 — Am I adding/removing ONE column, not the whole table?

DEFINITION

The 5-Question DDL Decision Tree (2/3)

→ Yes? → ALTER TABLE ADD/DROP COLUMN. The rest of the table stays intact.

Q4 — Do I need to EXPAND a type (VARCHAR(50) → VARCHAR(200))?

→ Yes? → ALTER COLUMN TYPE. Expanding is always safe; shrinking may fail if data is too long.

Q5 — Is this a throwaway test object that's safe to delete?

→ Yes? → DROP TABLE/SCHEMA IF EXISTS. If ANY production data lives here — STOP and back up first.

🚨 **Golden Rule:**

DEFINITION

The 5-Question DDL Decision Tree (3/3)

In production, every DDL change goes through a versioned migration file. No hand-typed ALTER on prod. Ever.

INTERVIEW

Q1 — DDL vs DML vs DCL vs TCL?

DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE — defines structure.

DML (Data Manipulation): INSERT, UPDATE, DELETE, SELECT — manipulates rows.

DCL (Data Control): GRANT, REVOKE — permissions.

TCL (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT — transaction boundaries.

Memory hook: DDL = 'Design', DML = 'Data', DCL = 'Control', TCL = 'Transaction'.

INTERVIEW

Q2 — DROP vs TRUNCATE vs DELETE — which one when?

DROP TABLE — removes table structure AND data. Unrecoverable without backup.

TRUNCATE TABLE — removes ALL data, keeps structure. Faster than DELETE, cannot be rolled back in some DBs, resets SERIAL counters.

DELETE FROM — removes rows matching WHERE clause (or all if no WHERE). Logged, rollback-able, slower on big tables.

Interview gotcha: TRUNCATE is DDL (auto-commit in some DBs); DELETE is DML (transactional). Know the difference.

INTERVIEW

Q3 — Why is CREATE TABLE IF NOT EXISTS a best practice?

It makes your DDL scripts idempotent — safe to run multiple times. If a teammate already ran the script on shared dev DB, the second run doesn't crash. Critical for CI/CD pipelines where migration scripts may rerun.

INTERVIEW

Q4 — What does CASCADE do in DROP SCHEMA?

DROP SCHEMA retail CASCADE deletes the schema AND every table/view/function inside it. Without CASCADE, PostgreSQL refuses to drop a non-empty schema — protecting you from accidentally nuking 47 tables with one command.

INTERVIEW

Q5 — Can I ALTER a column's type while the table has data?

Yes, but with restrictions. Widening is always safe (VARCHAR(50) → VARCHAR(200), INT → BIGINT). Narrowing fails if any row exceeds the new limit. Type change (VARCHAR → INTEGER) requires USING clause: ALTER COLUMN price TYPE INTEGER USING price::INTEGER.

INTERVIEW

Q6 — Why do production teams use migration files instead of raw ALTER?

Migrations give you: (1) version control — every change is a git commit; (2) rollback — each migration has an 'up' and 'down'; (3) audit trail — who changed what, when; (4) environment sync — dev, staging, prod stay in lockstep. Tools: Flyway (Java), Alembic (Python), Liquibase, Prisma Migrate, Knex (Node).

INTERVIEW

Q7 — What's a schema in PostgreSQL, and why use multiple schemas?

A schema is a namespace inside a database. In RetailMart V2: customers, products, sales, hr, support — 16 schemas. Benefits: logical grouping, permission isolation (grant access to 'hr' schema only to HR team), avoiding name collisions (sales.orders vs support.tickets_orders).

INTERVIEW

Q8 — What happens when you DROP a column that's referenced by a foreign key?

PostgreSQL refuses with: 'cannot drop column because other objects depend on it'. You must either DROP the FK constraint first, or use DROP COLUMN … CASCADE which will auto-drop the dependent constraint. CASCADE is dangerous — always prefer the explicit two-step approach.

INTERVIEW

Q9 — Is ALTER TABLE atomic? What happens if it fails midway?

In PostgreSQL, each ALTER TABLE statement runs inside an implicit transaction — if it fails, the table reverts to its previous state (no partial changes). BUT: long-running ALTER on a huge table can take ACCESS EXCLUSIVE lock, blocking all reads/writes. In production, use pg_repack or create-new-table-and-swap patterns for zero-downtime migrations.

INTERVIEW

Q10 — What's the difference between DROP DATABASE and DROP SCHEMA?

DROP DATABASE nukes an entire PostgreSQL database — all schemas, tables, users, everything. You must be connected to a DIFFERENT database to drop one (can't drop the DB you're connected to). DROP SCHEMA only removes a namespace inside the current database.

DEFINITION

10 Rapid-Fire Questions — 90 Seconds Total (1/3)

Close this notebook. Take a blank paper. Answer all 10 in 90 seconds. Score yourself — 8+ = confident, 5-7 = review Setup & Onboarding, under 5 = redo the whole day.

1.: Which DDL command adds a new column to an existing table?

2.: Is TRUNCATE reversible?

3.: What does IF EXISTS do in DROP TABLE IF EXISTS orders?

4.:

DEFINITION

10 Rapid-Fire Questions — 90 Seconds Total (2/3)

Write the SQL to rename a table 'orders' to 'invoices'.

5.: Can you drop a database you're currently connected to?

6.: What's the default SCHEMA in PostgreSQL if you don't specify one?

7.: Will ALTER TABLE … ALTER COLUMN price TYPE INTEGER work on a column with data '₹500'?

8.: Which is faster for removing all rows: DELETE or TRUNCATE?

DEFINITION

10 Rapid-Fire Questions — 90 Seconds Total (3/3)

9.: How do you check the structure of a table in PostgreSQL?

10.: What does CASCADE do when dropping a schema?

REVEAL

🎓 ANSWER KEY — Check Yourself (1/2)

1. ALTER TABLE … ADD COLUMN col TYPE;
2. No — TRUNCATE in PostgreSQL can be rolled back inside a transaction, but it's typically treated as DDL and can't be rolled back in other DBs (MySQL).
3. Prevents error if the table doesn't exist — makes the script safe to re-run.
4. ALTER TABLE orders RENAME TO invoices;
5. No — you must connect to a different database first (e.g., 'postgres' database).
6. public
7. No — '₹500' contains non-numeric characters. You need USING clause: ALTER COLUMN price TYPE INTEGER USING REPLACE(price, '₹', '')::INTEGER.
8. TRUNCATE — it doesn't scan rows; it just deallocates pages.
9. \d tablename (psql shortcut) OR SELECT * FROM information_schema.columns WHERE table_name = 'x';

REVEAL

🎓 ANSWER KEY — Check Yourself (2/2)

10. Drops the schema AND everything inside it (tables, views, functions) in one shot.

PRO TIP

PRO TIP (1/2)

Pass 1 — Read + Code Along (45 min)

Open your PostgreSQL terminal. Read each section, then actually TYPE and run every CREATE, ALTER, DROP command shown. Don't copy-paste — muscle memory matters.

Pass 2 — Recall without Notes (20 min)

Close the notebook. Try to write these from memory: (a) CREATE DATABASE syntax, (b) CREATE SCHEMA syntax, (c) all 4 forms of ALTER TABLE, (d) DROP TABLE with CASCADE. Open the notes only when you're stuck — that struggle is where learning happens.

Pass 3 — Teach It (15 min)

Explain to an imaginary junior (or a friend): 'What's the difference between DROP, TRUNCATE, and DELETE?' If you can teach it cleanly, you own it. If you stumble — you've found the weak spot. Re-read that section.

PRO TIP

PRO TIP (2/2)

🎯 Total time: 80 minutes. Do this and you will SAIL through earlier sessions.

CHALLENGE

CHALLENGE (1/3)

🔥 THE 6-STEP CHALLENGE

You have 30 minutes. No Googling syntax. Only your Setup & Onboarding notes.

Step 1 —: CREATE DATABASE my_retailmart (do this from the psql command line, not from inside another DB).

Step 2 —: Connect to my_retailmart. Create schemas: customers, products, sales.

Step 3 —:

CHALLENGE

CHALLENGE (2/3)

In customers schema, create table customers with: customer_id SERIAL PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), email VARCHAR(100), registration_date DATE.

Step 4 —: In products schema, create table products with: product_id SERIAL PRIMARY KEY, product_name VARCHAR(100), price NUMERIC(10,2), brand_id INT.

Step 5 —: ALTER customers table: ADD COLUMN phone VARCHAR(15), RENAME COLUMN registration_date TO signup_date.

Step 6 —: Run SELECT * FROM information_schema.tables WHERE table_schema IN ('customers','products','sales'); — you should see your 2 tables listed.

✅ **Bonus:**

CHALLENGE

CHALLENGE (3/3)

Write a single .sql file with all 6 steps so you can re-run it anytime. That's your first 'migration script.' Welcome to real-world database engineering.

REAL WORLD

REAL WORLD (1/2)

🏢 WHO USES DDL DAILY?

Every company that runs a database runs DDL. Here's who has legendary DDL migration stories:

- GitHub — ran a 2-week online migration on their main MySQL cluster (gh-ost tool, zero downtime)
- Instagram — sharded a single PostgreSQL into thousands of logical shards using schema-level DDL
- Stripe — every table change goes through a code-reviewed migration file; no hand-typed ALTER ever reaches prod
- Shopify — uses pt-online-schema-change for zero-downtime DDL on billions-of-rows tables
- Netflix — thousands of microservices each own their schema; DDL is deployed with the service code

REAL WORLD

REAL WORLD (2/2)

💡 The pattern: big companies treat DDL as sacred. A bad DDL deploy can take down the app for hours. Master it now and you'll be the engineer teammates trust with prod.

📱 WhatsApp Announcement

🎬 Tomorrow: Data Types Deep Dive

You learned to CREATE TABLE. But HOW do you pick the right type for each column?

- Should price be INT, NUMERIC, or FLOAT?
- Should phone be VARCHAR or INT?
- When do you use TEXT vs VARCHAR(N)?
- What's the difference between TIMESTAMP and TIMESTAMPTZ?

🔥 Choosing wrong = 10x slower queries + storage waste + data corruption bugs.
Choosing right = a database that scales to billions of rows.

DDL & Data Types turns you from 'I can create a table' → 'I can design a PRODUCTION-GRADE schema.'

YOUR SQL JOURNEY

- ✓ SQL Intro
- 2 **DDL & Queries** ← HERE
- 3 Data Types
- 4 Constraints
- 5 Filtering
- 8 Aggregates
- 9 Joins
- 15 Window Funcs
- 2 Transactions

Setup & Onboarding — DDL Commands Quick Reference

-- CREATE Commands

```
CREATE DATABASE db_name;
CREATE SCHEMA IF NOT EXISTS schema_name;
CREATE TABLE IF NOT EXISTS schema.tbl (
  col1 DATA_TYPE, col2 DATA_TYPE
);
```

-- DROP Commands

```
DROP TABLE IF EXISTS schema.tbl;
DROP SCHEMA IF EXISTS schema CASCADE;
DROP DATABASE IF EXISTS db_name;
```

-- ALTER TABLE

```
ALTER TABLE schema.tbl ADD COLUMN col TYPE;
ALTER TABLE schema.tbl DROP COLUMN col;
ALTER TABLE schema.tbl RENAME COLUMN a TO b;
ALTER TABLE schema.tbl ALTER COLUMN col TYPE x;
```